CM10194
Computer Systems Architectures

Operand Modes

Fabio Nemetz

Operand Modes and Instruction Sets

Summary

In this lecture:

- More about coding instructions: different modes for representing addresses in the store.

A real instruction set: MC68000

Example: the Motorola MC68000 architecture

Recall our *example of an addition operation that adds the 16-bit contents of a specified location (we chose $1202 = 1202 hexadecimal) to one of the eight data registers of this machine (we chose register 5).*

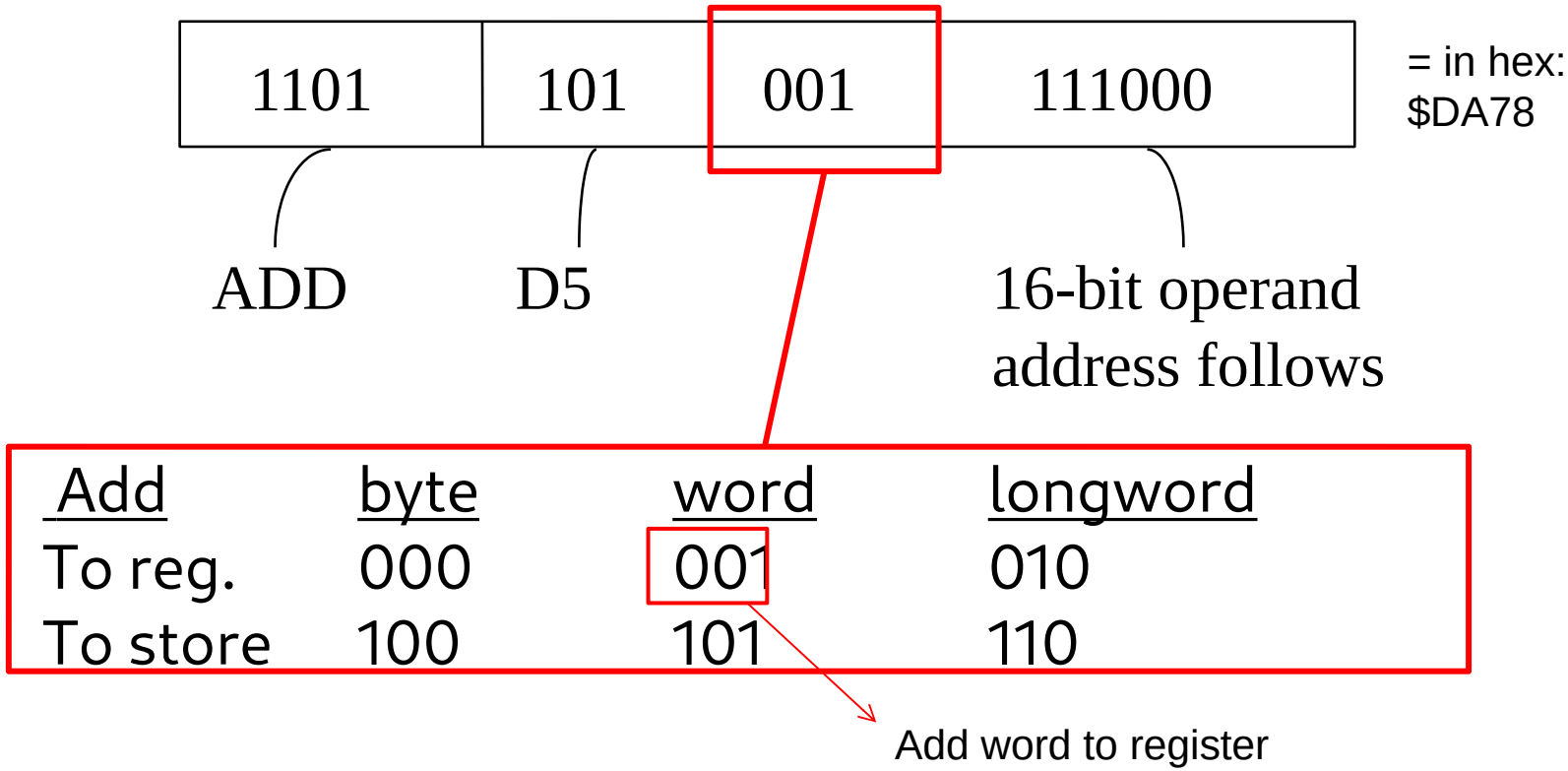
ADD $1202, D5

This instruction was stored in 4 bytes.

A real instruction set: MC68000

ADD $1202, D5

(a) The 2-byte operation (*control*) word* is:



***For the MC68000, a word means 16 bits – a word means "the natural chunk of data for this processor

A real instruction set: MC68000

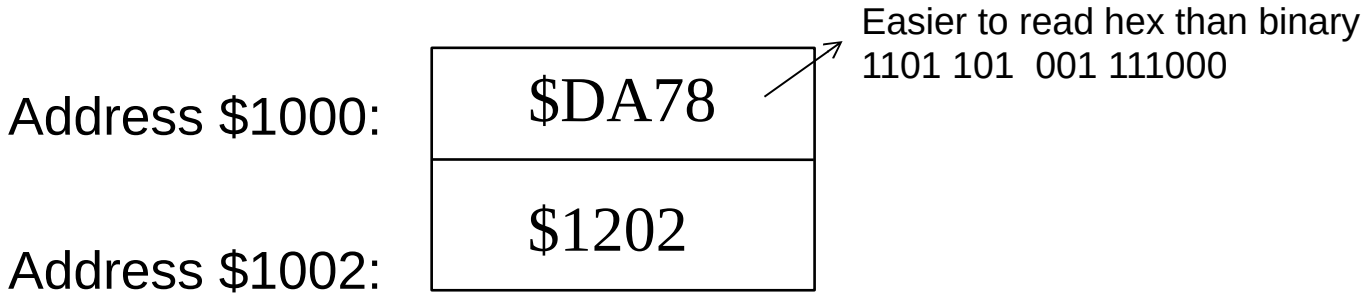
ADD $1202, D5

(b) The 2-byte operand address (*extension*) word is:

0001001000000010	= $1202

This is simply the store address $1202 in binary.

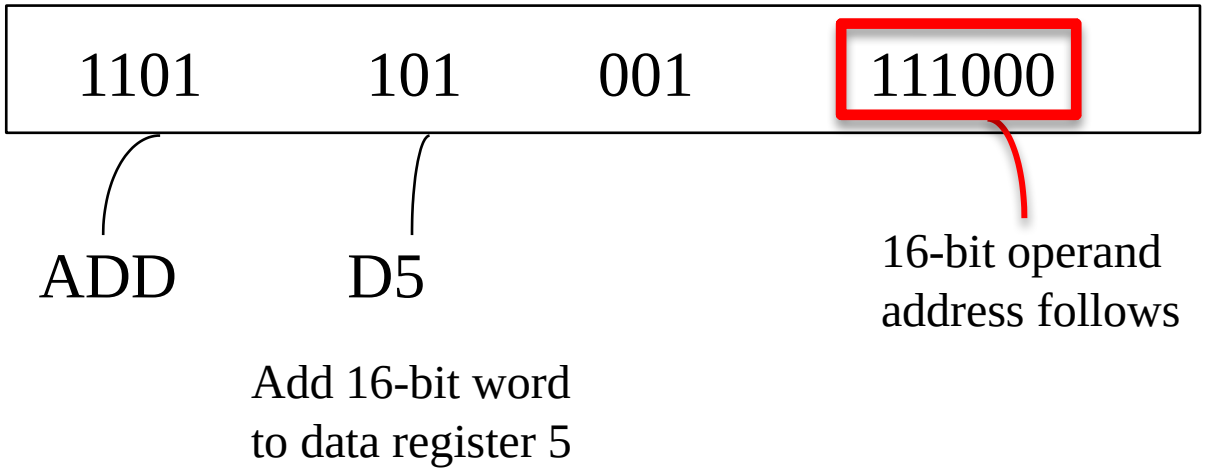
The four bytes (let us assume they are stored at the address $1000 hexadecimal) are thus (in hexadecimal):



Operand addressing modes

ADD $1202, D5

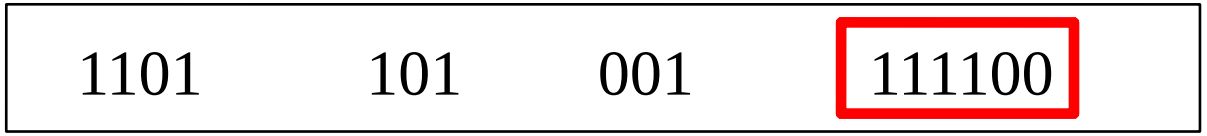
Recalling the 68000 addition operation, the *control* word was:



The last part (6 bits) represents a *mode* field that allows us to change the *way* an operand is accessed. In this case, it is by giving the *16-bit address* of the operand.

Operand addressing modes

If we had set this field of the instruction control word as follows:



then the 16-bit extension word that followed would have been treated *not* as the address of the operand, but as the *value* of the operand. So, if the extension word contained $1202 (hex.) as before, we would add the *value* 1202 (hex.) to data register 5.

Clearly there are potentially many different ways of using the bits of this field and this is important in determining the power of a computer's instructions.

Operand addressing modes

General structure of instructions

We can summarise what we have discovered by observing that an instruction in general has not just two, but three parts:

- Operation specification;
- Operand specification; and
- Mode specification (to determine how the operand is interpreted)

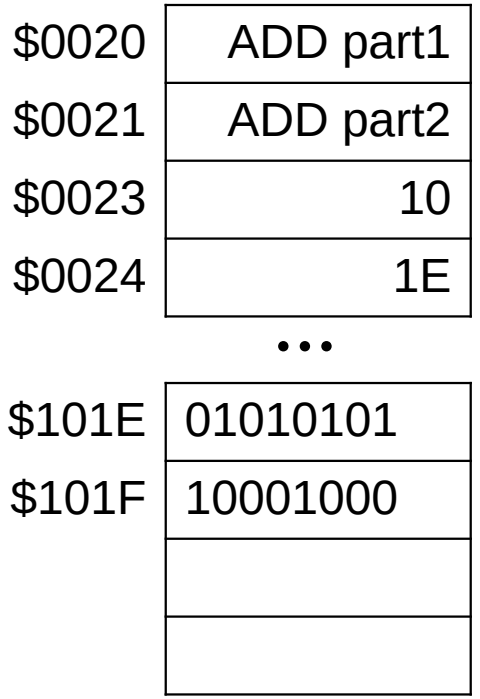
Operand addressing modes

Examples of different modes of operand access

- **Immediate:** the given value is used itself. Typically indicated with # in the MC68000.

- **Direct:** the value of the given address is used

Immediate	Direct
ADD **#**$101E, D1	ADD $101E, D1
D1=D1+$101E	D1=D1+mem($101E) (D1 = D1+ *$101E) (C-like)



Operand addressing modes

- **Indirect:** the operand is a location of store (or a register) which contains not data, but *another* address (where the true operand will be found);

  Typically indicated by placing the operand in parentheses

  e.g. ADD ($101E), D2 (in this case, add D2 with the content of the content of memory addressed by $101E)

Immediate	Direct	Indirect
ADD **#**$101E, D1	ADD $101E, D1	ADD ($101E), D1
D1=D1+$101E	D1=D1+mem($101E)	D1=D1+mem(mem ($101E))

$101E	01010101
$101F	10001000

Operand addressing modes

Homework:

- which address mode should perform faster?
- with an example, show why.

More Addressing Modes

- **We covered the 3 most common modes:**
  - Immediate
  - Direct
  - Indirect

- But there are more modes, depending on the processor
  - Indirect with displacement
  - Indirect with post-increment
  - Indirect with pre-decrement
  - Indirect with index
  - Even for PC (program counter)
    - PC with displacement
    - PC with index
  - (for more details see lecture notes)

Instruction Set

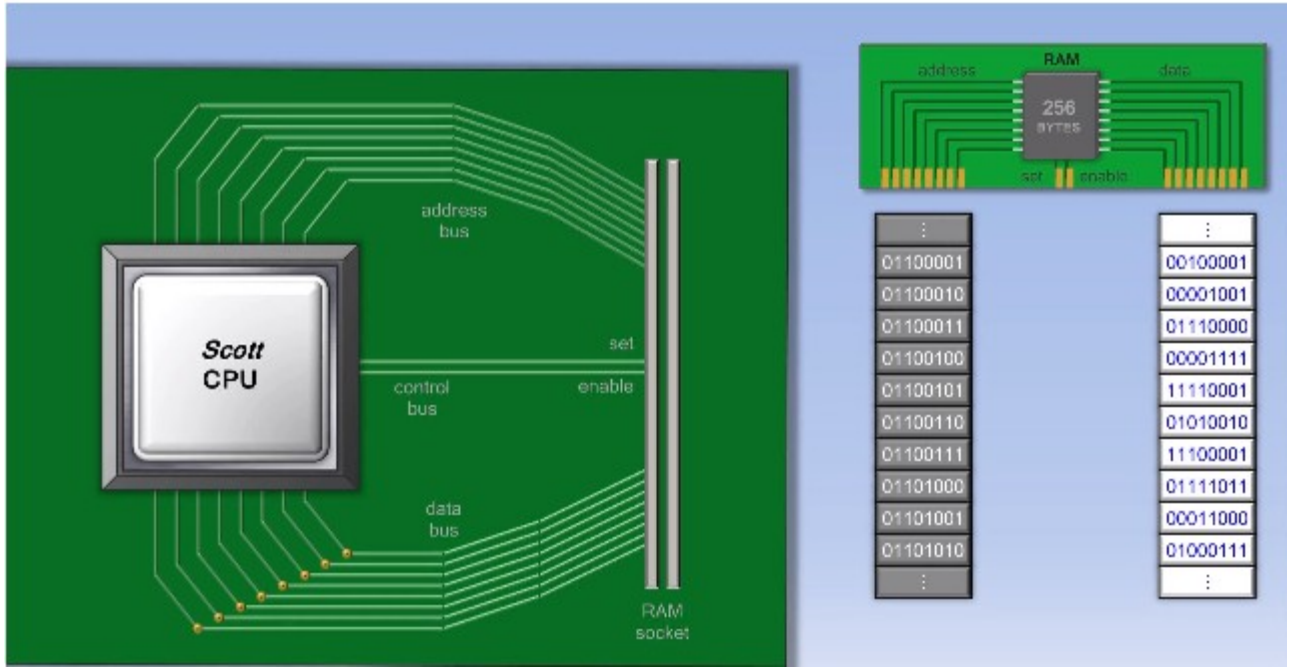
So far we've seen some instructions like:

LOAD and ADD

Other instructions:

- STORE or MOVE – store a value from a register in memory
- COMPARE – compare two values
- JUMP - jump to another address in memory (branch)
- JUMP IF Condition – JUMP if condition is true (conditional branch)

Video (to consolidate)



To recap: how the CPU works https://www.youtube.com/watch?v=IkdBs21HwF4

- the video makes a bit of a commercial to a book which I'm not endorsing.
- the processor of the example is the **6502** (used in the original Apple II)
- the example uses instructions **IN** and **OUT** in a very simplified way – we'll talk about them later when we discuss **Input and Output**
- **watch up to 11mins30s – we'll cover the rest later in the unit**

Summary

In this lecture we have introduced:

- The idea of a *address modes.*

Next lecture

- Design philosophies for instruction sets: RISC vs. CISC
- Control instructions:
  - unconditional branching
  - conditional branching and
  - subroutines
- Faster execution via Instruction level parallelism
  - pipelining and superscalar architectures.